EBOOK — Controle Financeiro do Zero

Tutorial guiado, microetapa por microetapa.

> Projeto didático: um site responsivo de controle financeiro pessoal (receitas, despesas e saldo), construído do zero, ensinando programação para iniciantes absolutos.

Como funciona este tutorial

- Avançamos **uma microetapa por vez**: objetivo → implementação → explicação → teste → resultado.
- Linguagem acessível: cada conceito novo é explicado primeiro em português simples.
- Usa apenas HTML, CSS e JavaScript puro — sem instalar nada além de um editor (VSCode, opcional).

Sumário

1. Fase 1 — Estrutura e interface básica (4 microetapas)
2. Fase 2 — Cadastro de transações (6 microetapas)
3. Bugs corrigidos (saldo após F5 e transação sem valor)
4. GitHub: subindo o projeto para a nuvem
5. Glossário: conceitos aprendidos
6. Próximos passos

FASE 1 — ESTRUTURA E INTERFACE BÁSICA

Microetapa 1 — A estrutura de arquivos

Objetivo: criar a "casa" do projeto: uma pasta com 4 arquivos.

Arquivo	Função
index.html	Estrutura da página (textos, campos, botões)
style.css	Visual (cores, espaçamento, responsividade)
app.js	Lógica (somar, guardar, mostrar dados)
PROGRESSO.md	Memória do projeto (diário de desenvolvimento)

Conceito novo — extensão de arquivo: é o final do nome (`html`, `css`, `js`). Diz ao computador o tipo do arquivo e qual programa sabe abrir. O `index.html` é sempre a porta de entrada de um site.

Microetapa 2 — A base do HTML

Objetivo: montar o esqueleto: cabeçalho e área principal.

```
<body>
  <header>
    <h1>Meu Controle Financeiro</h1>
  </header>
  <main>
    <p>Aqui vão ficar seus registros de receitas e despesas.</p>
  </main>
</body>
```

- `<header>` → topo da página (cabeçalho). Tag semântica.
- `<h1>` → título grande.
- `<main>` → área principal do conteúdo.
- `<p>` → parágrafo.

Conceito novo — tags em pares: `<header>` abre e `</header>` fecha. Como caixas: o que está dentro pertence à caixa, e caixas podem conter outras (o `<h1>` dentro do `<header>`).

Microetapa 3 — O primeiro CSS

Objetivo: dar cores e organização; funcionar em PC e celular.

Conexão ao HTML: `<link rel="stylesheet" href="style.css">`

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background-color: #f4f4f9;
  color: #333;
}

header {
  background-color: #4a90e2;
  color: white;
  padding: 20px;
  text-align: center;
}

@media (min-width: 768px) {
  main {
```

```
    max-width: 700px;
    margin: 0 auto;
  }
}
```

- body → regras gerais. `margin: 0` remove espaços automáticos; `#f4f4f9` é **código de cor hexadecimal** (começa com #).
- `padding` → espaço interno da caixa.
- `text-align: center` → centraliza texto.

Conceito novo — media query (responsividade): `@media (min-width: 768px)` vale somente quando a tela tem pelo menos 768px. No celular o conteúdo ocupa a largura toda; no PC fica estreito e centralizado (`margin: 0 auto`).

Microetapa 4 — O primeiro JavaScript

Objetivo: JS escreve o saldo na tela. Vale para o projeto inteiro: **HTML define o lugar, CSS enfeita, JS comanda.**

```
<p>R$ <span id="saldo"></span></p>
<script src="app.js"></script>
```

```
var saldo = 0;

function exibirSaldo() {
  document.getElementById("saldo").textContent = "R$ " + saldo.toFixed(2).replace(".",
}

exibirSaldo();
```

- `<span id="saldo">` → caixinha com um **endereço** (`id`) que o JS acha.
- `<script>` → carrega o JS (como o `<link>` carrega o CSS).
- `var saldo = 0;` → **variável**: caixa na memória que guarda um valor.
- `function exibirSaldo() {...}` → **função**: bloco que executa quando chamado.
- `document.getElementById("saldo")` → o JS pega o elemento pelo id.
- `.textContent` → substitui o texto dentro do elemento.
- `toFixed(2)` → 2 casas decimais. `.replace(".", ",")` → formato brasileiro.

- `exibirSaldo();` → **chamada da função** — sem ela, a função não roda.

FASE 2 — CADASTRO DE TRANSAÇÕES

Microetapa 1 — O formulário de receita

Objetivo: criar a tela para digitar descrição e valor de uma receita.

```
<form id="form-receita">
  <label for="descricao">Descrição</label>
  <input type="text" id="descricao" placeholder="Ex: Salário">
  <label for="valor">Valor (R$)</label>
  <input type="number" id="valor" step="0.01" min="0">
  <button type="submit">Adicionar</button>
</form>
```

- `<form>` → formulário; ganhou `id` para o JS achar.
- `<label for="...">` → etiqueta ligada ao campo de mesmo `id`.
- `input type="text"` → campo de texto; `placeholder` = sugestão que some ao digitar.
- `input type="number"` → só números; `step="0.01"` = centavos; `min="0"` barra negativos.
- `button type="submit"` → botão que envia; o JS vai interceptar.

Microetapa 1b — O visual dark + laranja

Objetivo: tema escuro com laranja, inspirado no Meu Planner Financeiro.

Paleta: fundo `#111827`, cartões `#1f2937`, destaque `#f97316`, texto `#f9fafb`.

- **Dark mode cria profundidade:** o que está por cima é mais claro que o fundo.
- **Seletor combinado** `header h1` = o `<h1>` dentro do `<header>`.
- `border-radius` nos `<section>` → cantos arredondados viram cartões.
- Contraste: campo escuro com texto claro (legibilidade).
- `#saldo` com `font-size: 2em` e `font-weight: bold` → saldo em destaque.

Microetapa 2 — O botão Adicionar funciona

Objetivo: o clique pega o valor digitado, soma ao saldo e mostra na tela.

```
document.getElementById("form-receita").addEventListener("submit", function (evento) {
  evento.preventDefault();

  var descricao = document.getElementById("descricao").value;
  var valor = Number(document.getElementById("valor").value);

  saldo = saldo + valor;
  exibirSaldo();
});
```

- `addEventListener("submit", ...)` → o JS **escuta** o evento de envio.
- `evento.preventDefault()` → **impede recarregar a página** (comportamento padrão).
- `.value` → lê o que foi digitado.
- `Number(...)` → converte texto em número: `"250" + 100` não soma, vira `"250100"`.
- `var descricao` → guardada para a próxima etapa (lista de transações).

Conceito novo — evento: um sinal do navegador ("alguém clicou"); o JS escuta e reage.

Microetapa 3 — O formulário de despesa

Objetivo: cadastrar despesas — mesma estrutura da receita, mas **subtrai** valor.

Os campos usam **ids diferentes** (`descricao-despesa`, `valor-despesa`) porque **ids devem ser únicos** na página.

```
document.getElementById("form-despesa").addEventListener("submit", function (evento) {
  evento.preventDefault();
  var valor = Number(document.getElementById("valor-despesa").value);
  saldo = saldo - valor;
  exibirSaldo();
});
```

Lição sobre padrão: copiamos uma estrutura parecida e ajustamos o essencial (o `id` e o sinal `-`). Mais adiante vamos reduzir a repetição.

Microetapa 3b — Salvando com localStorage

Objetivo: os dados sobrevivem ao F5, sem servidor.

```
var saldo = Number(localStorage.getItem("saldo")) || 0;

function exibirSaldo() {
  document.getElementById("saldo").textContent = "R$ " + saldo.toFixed(2).replace(".",
  localStorage.setItem("saldo", saldo);
}
```

- `localStorage.getItem("saldo")` → lê da "gaveta" do navegador (texto ou `null`).
- `Number(...)` → converte de volta para número (a gaveta só guarda texto).
- `|| 0` → "ou então 0": sem nada guardado, começa em 0.
- `localStorage.setItem("saldo", saldo)` → grava; ficou dentro de `exibirSaldo()`, que roda a cada mudança → sempre que muda, salva (um só lugar para salvar evita bugs).

Conceito novo — persistência: guardar informações para além do momento em que o programa roda.

BUG CORRIGIDO — saldo não aparecia após F5

Sintoma: após atualizar, o saldo sumia; ao fazer uma transação nova, somava com o valor antigo.

Diagnóstico: a gaveta (localStorage) sempre funcionou. Faltava **pintar o número na tela ao abrir a página**: a chamada inicial `exibirSaldo();` no final do `app.js` havia sido perdida.

Correção: chamada `exibirSaldo();` adicionada no final do arquivo.

Lição: um código pode estar funcionando (guardando certo) mas não fazer o que esperamos (mostrar). Por isso testamos sempre — é assim que se depura em projetos reais.

MICROETAPA 4 — O EXTRATO (ARRAY E JSON)

Objetivo: cada transação aparece na tela como item do Extrato, e a lista inteira é salva na gaveta.

```
<section>
  <h2>Extrato</h2>
  <ul id="lista-transacoes"></ul>
</section>
```

```
var transacoes = JSON.parse(localStorage.getItem("transacoes")) || [];

function salvarTransacoes() {
  localStorage.setItem("transacoes", JSON.stringify(transacoes));
}

function exibirTransacoes() {
  var lista = document.getElementById("lista-transacoes");
  lista.textContent = "";

  for (var i = 0; i < transacoes.length; i++) {
    var item = document.createElement("li");
    var transacao = transacoes[i];
    item.textContent = transacao.descricao + " — R$ " + transacao.valor.toFixed(2).repl
    lista.appendChild(item);
  }
}
```

Conceitos novos:

- **Array (lista) `[ ]`:** guarda vários valores em fila.
- **Objeto `{ chave: valor }`:** agrupa informações (descrição + valor).
- **`push(...)`:** adiciona item no final da lista.
- **Laço `for`:** `for (var i = 0; i < transacoes.length; i++)` percorre a lista; `transacoes[i]` é o item da vez; `length` = tamanho.
- **`createElement("li")`:** cria um item (ainda fora da tela).
- **`lista.appendChild(item)`:** anexa o item criado dentro da lista.

- **JSON:** a gaveta só guarda texto. `JSON.stringify` vira texto (salvar); `JSON.parse` volta para lista (ler).
- **`lista.textContent = ""`:** limpa antes de redesenhar (senão a lista dobrava).

🎉 Fim da Fase 2: o app soma, subtrai, mostra o histórico e não perde nada.

BUG CORRIGIDO — transação sem valor entrava no extrato

Sintoma: clicar em "Adicionar" com valor vazio criava a transação mesmo assim.

Diagnóstico: `Number("")` vira `0`, e não havia checagem antes de salvar. Campos de formulário sugerem, mas não bloqueiam — a proteção de verdade é o código.

Correção (validação), nos dois formulários:

```
if (valor <= 0) {
  alert("Digite um valor maior que zero.");
  return;
}
```

Conceitos novos: `if (condição)` = o "se" da programação; `alert("...")` = janelinha de aviso; `return;` = sai da função na hora, ignorando o resto (o "ponto de interdição").

Lição: todo dado que entra precisa ser **validado** antes de ser usado — base de segurança em qualquer app.

GITHUB — SUBINDO O PROJETO PARA A NUVEM

Git = a "máquina do tempo" (registra versões, dá para voltar no tempo). **GitHub** = serviço grátis que guarda o repositório na nuvem (backup + compartilhamento).

Passos feitos:

- `git init` → transformou a pasta em repositório.
- `git config user.name` / `user.email` → identidade para os commits.
- `git add .` → "empacotou" os arquivos (staging).
- `git commit -m "..."` → tirou a "fotografia" (versão salva).
- `git branch -m main` → branch no padrão do GitHub.
- `git remote add origin <url>` → conectou com o repositório na nuvem.
- `git push -u origin main` → **enviou tudo.**

Guardar uma etapa nova: `git add .` → `git commit -m "mensagem"` → `git push`.

Conceitos: commit = versão registrada; branch = linha de desenvolvimento (main); remote = endereço da nuvem; push = enviar; repositório = pasta + máquina do tempo.

Link do projeto: https://github.com/taylorhdrescher/Controle-Financeiro (botão "commits" mostra a linha do tempo).

GLOSSÁRIO — Conceitos aprendidos

Conceito	O que é	Onde usamos
Extensão de arquivo	Final do nome que identifica o tipo	.html, .css, .js
Tag em par	Elemento HTML que abre e fecha	...
Aninhamento	Caixas dentro de caixas	<h1> dentro de <header>
Variável	Caixa na memória que guarda valor	var saldo
Função	Bloco que executa quando chamado	function exibirSaldo()

Id	Endereço único de um elemento	id="saldo"
Evento	Sinal do navegador (clique etc.)	addEventListener("submit")
preventDefault	Impede o comportamento padrão	parar recarga do form
.value	Lê o que foi digitado no campo	getElementById("valor").value
Number()	Converte texto em número	Number("250") → 250
toFixed(2)	Força 2 casas decimais	R$ 0,00
Media query	Regra CSS condicionada ao tamanho da tela	responsividade
localStorage	Gaveta de memória do navegador	salvar entre F5
Persistência	Guardar dados além do programa rodando	saldo não some
Array	Lista que guarda vários valores	var transacoes = [...]
Objeto	Agrupa informações sobre a mesma coisa	{ descricao, valor }
push()	Adiciona item ao final da lista	transacoes.push(...)
length	Tamanho de uma lista	transacoes.length
Laço for	Repete um bloco várias vezes	percorrer a lista
createElement	Cria um elemento no JS	criar
appendChild	Anexa o elemento criado na página	colocar na lista
JSON	Formato texto que representa dados	salvar a lista na gaveta
Validação	Checar o dado antes de usar	if (valor <= 0)
if	O "se" da programação	condição
alert()	Janelinha de aviso do navegador	"Digite um valor maior..."
return	Sai da função na hora	interromper envio inválido

Git / commit	Máquina do tempo / versão registrada	backup do projeto
GitHub / push	Nuvem / enviar para a nuvem	repositório online

PRÓXIMOS PASSOS

- Fase 3: Categorias (moradia, alimentação, lazer...) — organizando as despesas.
- Fase 4: Resumo do mês com cores (verde receita / vermelho despesa).
- Fase 5: Diferenciais (importação de extrato CSV ou Open Finance simulado).